

29 | 安全提升：如何解决 GitOps 的密钥存储问题？

王炜 · 云原生架构与GitOps实战



你好，我是王炜。

在上节课，我为你介绍了在 GitOps 工作流中需要特别关注的安全问题。其中，由于 Git 仓库是 GitOps 工作流的唯一可信源，同时也包含了 Kubernetes 对象以及机密信息，所以，我们首先需要关注它的安全问题。

在实际的业务场景下，出于安全需要，Git 仓库往往会包含下面这些机密信息。

1. 镜像拉取凭据的 Secret 对象，它可以为集群提供拉取镜像的权限。
2. 外部数据库连接信息。
3. 外部中间件如 MQ 连接信息。
4. 第三方服务的 API KEY，例如云厂商和短信服务商。

在 GitOps 工作流中，这些机密信息都会通过 Kubernetes 对象存放在 Git 仓库，在大部分情况下它们是 Configmap 或 Secret 对象。虽然 Secret 被设计为存储 Kubernetes 的机密信息，但它只是 Base64 编码后的结果，不具备加密性质，这也就意味着机密信息完全是以明文的方式暴露的，这是非常不安全的。所以，我们有必要对这些机密信息进行加密。

这节课，我们就来学习加密 Git 仓库中机密信息的方法，进一步提升 GitOps 的安全性。

在开始之前，你需要在本地的 Kind 集群安装好 ArgoCD，然后克隆 [🔗 示例仓库](#)，并将它推送到你的 Git 仓库中。

GitOps 密钥管理方案

在 GitOps 工作流中，常用的密钥管理方案有下面三种。

1. [🔗 Sealed-Secrets](#)
2. [🔗 External-Secrets](#)
3. [🔗 Vault](#)

其中，Sealed-Secrets 在易用性方面比较大的优势，社区活跃度也比较高，也更容易和 GitOps 工作流结合，我会在这节课做重点介绍。它的工作原理也非常简单，它利用非对称加密算法对 Secret 对象进行加密，使用的时候在集群内自动进行解密，这样就可以将加密后的密钥安全地存储在 Git 仓库中。

External-Secrets 需要外部密钥管理服务的支持，例如 AWS Secrets Manager、Google Secrets Manager、Azure Key Vault 等，如果在你的项目里用到了这些密钥管理服务，可以考虑使用它。

Vault 是 HashiCorp 开源的一款密钥管理工具，要将它和 ArgoCD 结合使用需要额外的插件，配置起来比较繁琐。

安装 Sealed-Secrets

接下来，我们来看看如何把 Sealed-Secrets 集成到 GitOps 工作流中，并将它结合 ArgoCD 一起使用。

Sealed-Secrets 分为两部分，本地的 CLI 工具和运行在集群的控制器，在使用之前我们需要先安装它们。

安装 CLI 工具

kubeseal 命令行工具是本地和集群 Sealed-Secrets 服务交互的工具，我们需要用它来加密机密信息。

我们以 MacOS 系统为例，你可以使用 Brew 进行安装。

```
1 $ brew install kubeseal
```

 复制代码

如果你使用的是 Linux 或 Windows，可以在 [🔗 这个链接](#) 将可执行文件下载到本地。

安装 Controller 控制器

Sealed-Secrets 控制器负责对加密的信息进行解密，并生成 Kubernetes 原生的 Secret 对象。这里我推荐你以 Helm 的方式来安装它。

首先，添加 Sealed-Secrets 的 Helm 仓库。

```
1 $ helm repo add sealed-secrets https://bitnami-labs.github.io/sealed-secrets
```

 复制代码

然后，通过 helm install 命令来安装它。

 复制代码

```
1 $ helm install sealed-secrets -n kube-system --set-string fullnameOverride=sealed
```

接下来，等待工作负载处于 Ready 状态。

 复制代码

```
1 $ kubectl wait deployment -n kube-system sealed-secrets-controller --for conditic
```

示例应用介绍

在这节课，我编写了一个简单的 Deployment 工作负载，它包含我要介绍的两种密钥类型，分别是**镜像拉取凭据**和 **Kubernetes Secret 对象**。

工作负载的 Deployment 内容如下。

 复制代码

```
1 apiVersion: apps/v1
2 kind: Deployment
3 metadata:
4   name: sample-spring
5 spec:
6   .....
7   spec:
8     imagePullSecrets:
9     - name: github-regcred
10    containers:
11    - name: sample-spring
12      image: ghcr.io/lyzhang1999/sample-kotlin-spring:latest
13      ports:
14      - containerPort: 8080
15        name: http
16      env:
17      - name: PASSWORD
18        valueFrom:
19          secretKeyRef:
20            key: password
21            name: sample-secret
```

其中，我将这节课的示例应用镜像存放到了 GitHub Package 仓库中，也就是域名为 ghcr.io 的镜像仓库中，同时设置了私有仓库类型。在没有向 Kubernetes 集群提供拉取凭据的情况下，是无法拉取镜像的，这意味着直接将这个工作负载部署到集群将得到 ImagePullBackOff 的事件。

imagePullSecret 是镜像拉取凭据，这个凭据我们将会在稍后通过 kubeseal 创建。

此外，我还为工作负载配置了 Env 环境变量，它的值来源于名为 sample-secret 的 Secret 对象。这个 Secret 对象我们也会在稍后通过 kubeseal 来创建。

创建 ArgoCD 应用

为了让 Sealed-Secrets 和 GitOps 工作流结合，我们首先需要创建 ArgoCD 应用。

在将 [🔗 示例应用仓库](#) 克隆到本地并推送到你自己的 Git 仓库之后，你需要修改 sealed-secret/application.yaml 文件，将 repoURL 替换为你的 Git 仓库地址。

 复制代码

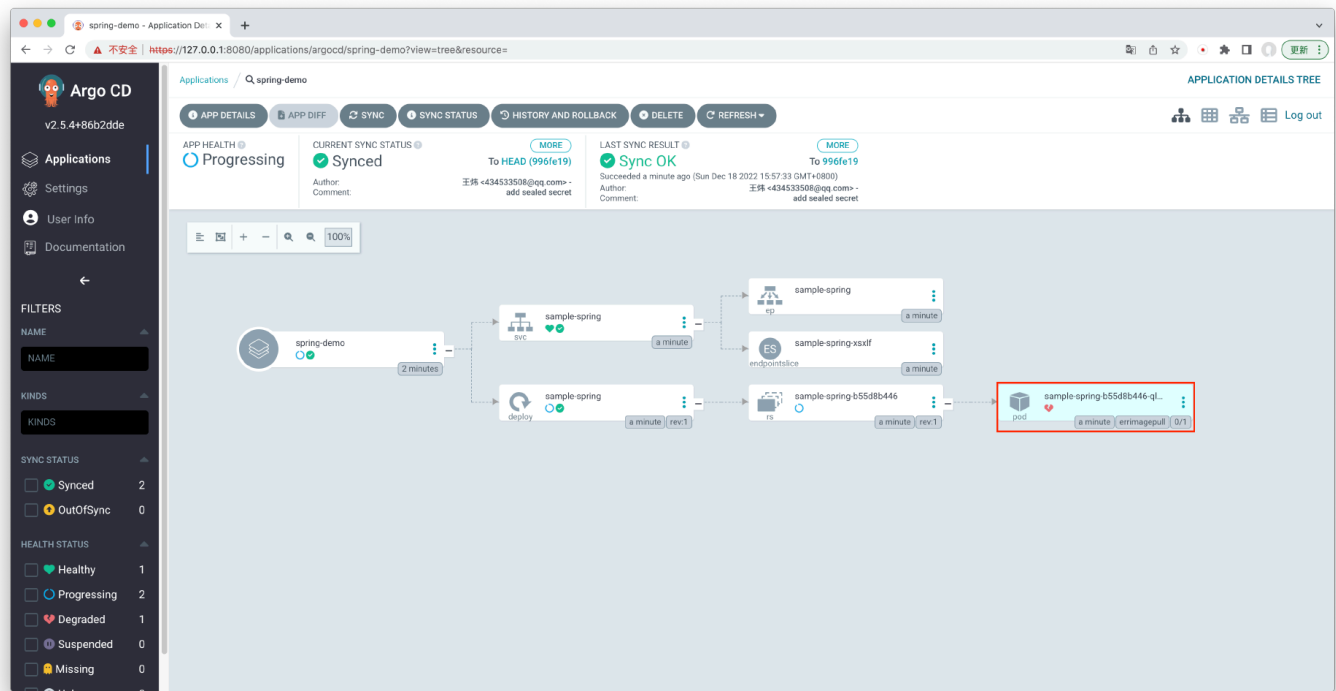
```
1 apiVersion: argoproj.io/v1alpha1
2 kind: Application
3 metadata:
4   name: spring-demo
5 spec:
6   project: default
7   source:
8     # 替换为你的 Git 仓库地址，注意将仓库权限设置为公开，并使用 https 协议
9     repoURL: https://github.com/lyzhang1999/kubernetes-example.git
```

然后，进入 sealed-secret 目录，并通过 kubectl apply 命令将它应用到集群内。

 复制代码

```
1 $ cd sealed-secret
2 $ kubectl apply -f application.yaml
```

现在，你应该能在 ArgoCD 的控制台看到这个应用，如下图所示。



要注意的是，在进入 ArgoCD 控制台之前需要进行端口转发操作，并获取 ArgoCD admin 登录密码，具体操作你可以参考 [第 22 讲](#)。

但由于我们并没有为集群提供 imagePullSecret，所以 Kubernetes 集群无法拉取镜像，你会看到右下角的 Pod 抛出了 ImagePullBackOff 的错误。

创建加密后的 Secret 对象

接下来，我们对两种 Secret 对象进行加密，它们分别是镜像拉取凭据和普通 Secret 对象。然后我们要将加密后的 Secret 对象推送到 Git 仓库中，以便 ArgoCD 将它一并部署到集群内。

创建 Image Pull Secret 对象

为了让 Kubernetes 集群能够顺利拉取镜像，我们首先来为集群创建镜像拉取凭据。

在这节课的示例应用中，我已经将镜像拉取凭据的原始 Kubernetes Secret 对象存放在了 sealed-secret/image-pull-secret.yaml 文件中。

 复制代码

```
1 kind: Secret
2 type: kubernetes.io/dockerconfigjson
3 apiVersion: v1
4 metadata:
5   name: github-regcred
6 data:
7   .dockerconfigjson: eyJhdXRocyI6eyJnaGNyLmlvIjp7InVzZXJuYW1lIjoibHl6aGFuZzE5OTki
8
```

在 [🔗 上一讲](#) 我们提到，这个 Secret 对象实际上只经过 Base64 编码，它并不安全。你可以尝试将它进行 Base64 解码，这样就能够得到我的 GitHub Token 了，但它只有拉取镜像的权限。

接下来，进入示例应用的 sealed-secret 目录，我们来尝试创建加密后的 Secret 对象。你可以使用 kubeseal 命令来创建。

 复制代码

```
1 $ kubeseal -f image-pull-secret.yaml -w manifest/image-pull-sealed-secret.yaml --
```

简单解释一下这个命令。首先，-f 参数指定了原始 Secret 对象文件，也就是 image-pull-secret.yaml。-w 参数表示将加密后的 Secret 对象写入 manifest 目录的 image-pull-sealed-secret.yaml 文件内，这样 ArgoCD 就可以将它一并部署到集群内。--scope 参数表示加密后的 Secret 对象可以在集群的任何命名空间下使用。

然后，你可以查看 manifest/image-pull-sealed-secret.yaml 文件，加密后的 Secret 对象如下。

 复制代码

```
1 apiVersion: bitnami.com/v1alpha1
```

```
2 kind: SealedSecret
3 metadata:
4   annotations:
5     sealedsecrets.bitnami.com/cluster-wide: "true"
6   creationTimestamp: null
7   name: github-regcred
8 spec:
9   encryptedData:
10     .dockerconfigjson: AgBgEU0xC1i2AuZJ2LzPiSfYblycy71NGv1SapA46ugFlWyKRuUg+WQGDH
11   template:
12     metadata:
13       annotations:
14         sealedsecrets.bitnami.com/cluster-wide: "true"
15       creationTimestamp: null
16       name: github-regcred
17     type: kubernetes.io/dockerconfigjson
18
```

从文件的内容我们可以看出，原始的 Secret 对象被转化为了 SealedSecret 对象，并且增加了 encryptedData 字段，它可以存放加密后的内容。

创建 Secret 对象

除了创建镜像凭据以外，我们还需要创建为工作负载提供密码的 Secret 对象。我将原始的 Secret 对象存放在了 sealed-secret/sample-secret.yaml 文件下。

 复制代码

```
1 apiVersion: v1
2 kind: Secret
3 metadata:
4   name: sample-secret
5 data:
6   password: YWRtaW4K
```

这个 Secret 对象非常简单，password 字段记录了我们提供的机密信息，解码后实际上是 admin 字符串。

接下来，我们创建加密后的 Secret 对象。同样地，使用 kubeseal 命令来创建。

[📄 复制代码](#)

```
1 $ kubeseal -f sample-secret.yaml -w manifest/sample-sealed-secret.yaml --scope cl
```

运行命令后，在 manifest 目录下生成 sample-sealed-secret.yaml 文件，它包含加密后的 Secret 内容。

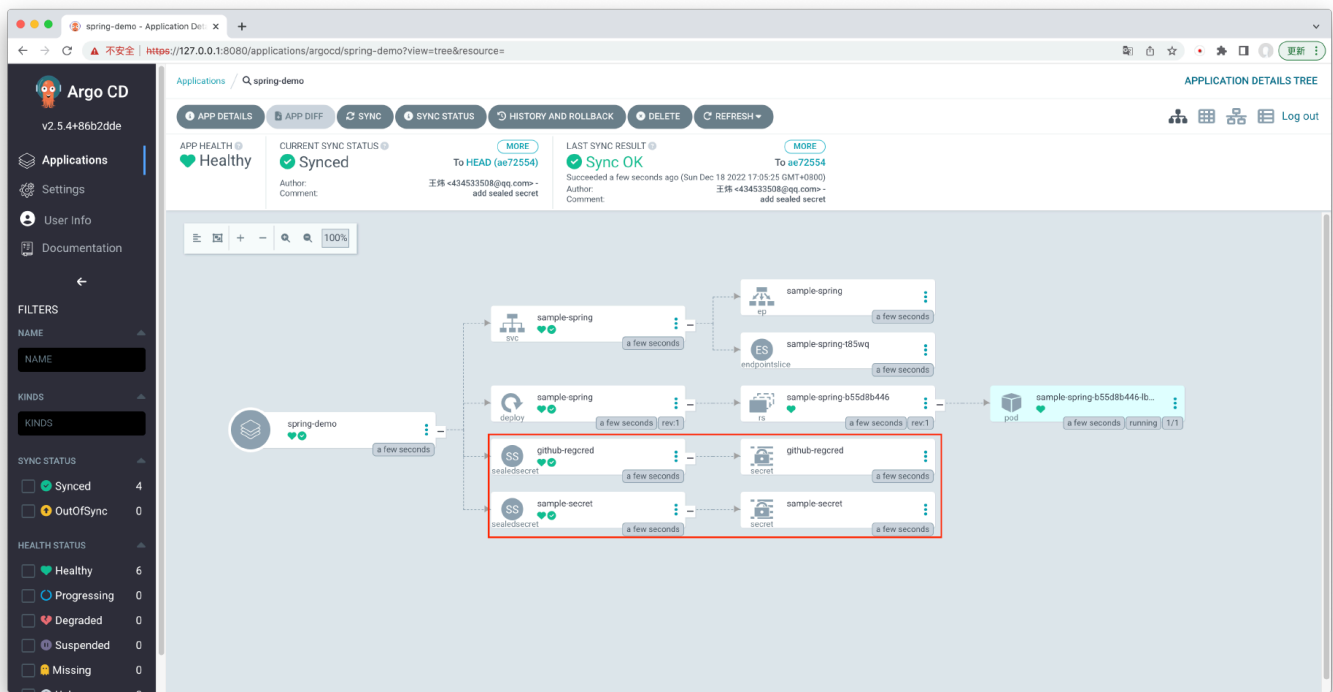
推送到代码仓库

现在，我们将刚才创建的加密后的文件推送到 Git 仓库中。

[📄 复制代码](#)

```
1 $ git add .  
2 $ git commit -a -m 'add secret'  
3 $ git push origin main
```

进入 ArgoCD 控制台的应用详情，手动点击“SYNC”按钮同步刚才我们新增的 Secret 对象。现在，你应该能看到应用状态变成了 Healthy 健康状态，并且 Sealed-Secret 控制器对刚才创建的 SealedSecret 对象进行了解密，还重新创建了原始的 Kubernetes Secret 对象以供 Deployment 工作负载使用，如下图所示。



现在，镜像已经顺利被拉取下来，工作负载也已经正常启动了。

验证 Secret

除了镜像凭据以外，我们还创建了为工作负载提供密码的 Secret 对象，它是通过 Env 环境变量注入到 Pod 中的。接下来，我们来验证应用是否能够顺利获取到来自 Secret 对象提供的密码。

首先，对示例应用进行端口转发操作。

复制代码

```
1 $ kubectl port-forward svc/sample-spring 8081:8080 -n secret-demo
```

然后，打开一个新的命令行窗口，并访问示例应用的获取环境变量接口。

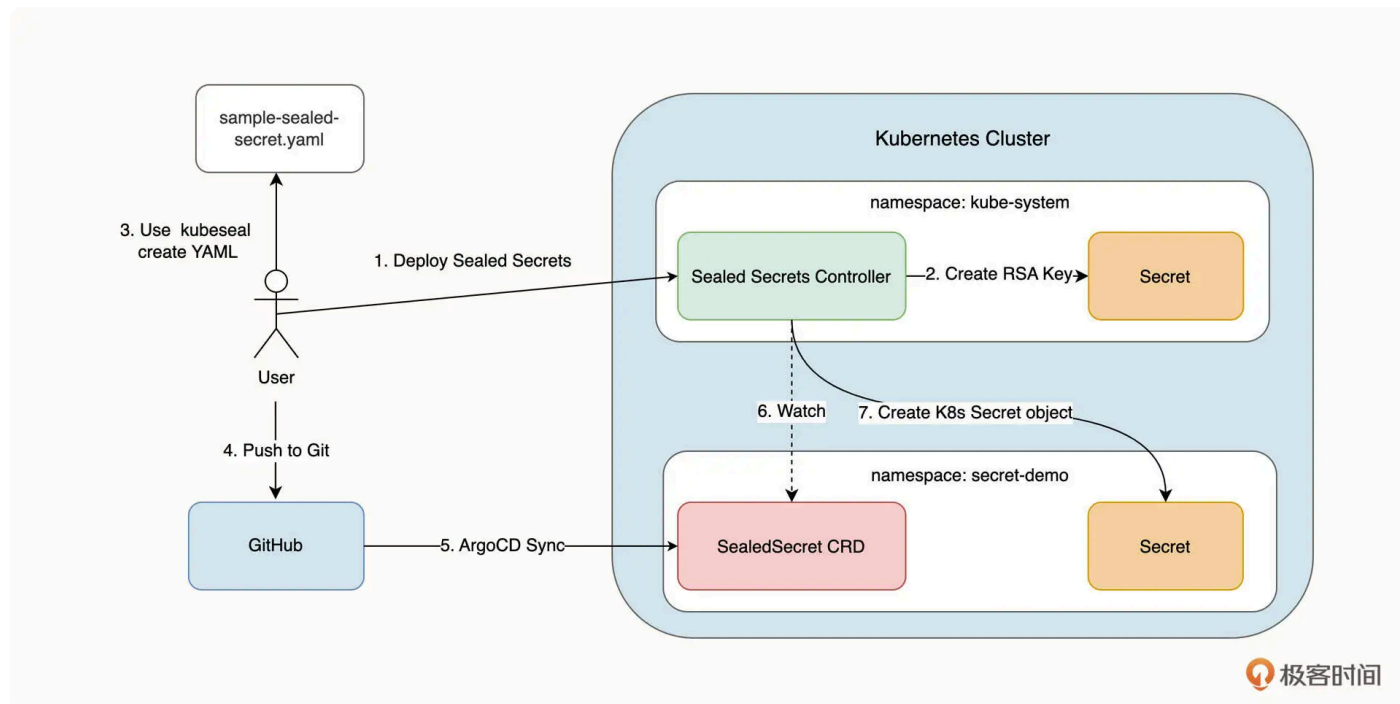
```
1 $ curl http://localhost:8081/actuator/env/PASSWORD
2
3 {"property":{"source":"systemEnvironment","value":"*****"},"activeProfiles":[],"
```

从返回结果我们可以发现，应用已经成功获取到了 PASSWORD 环境变量，这说明 Sealed-Secret 控制器也已经生成了原始的 Secret 对象。

通过上面的实验我们知道，在 GitOps 工作流中，当我们在集群内安装了 Sealed-Secret 控制器之后，只需要在 Git 仓库存储加密后的 SealedSecret 对象即可，不再需要存储原始的 Kubernetes Secret 对象了。

原理解析

Sealed-Secret 的整体工作原理比较简单，下面，我画了一张架构图来帮助你理解整个过程。



你可以按照架构图上的序号来依次理解每一个步骤。

由于加解密过程涉及到非对称加密，所以当我们在集群内安装 Sealed-Secret 控制器时，控制器会在集群范围内查找私钥 / 公钥对，如果没找到，则会生成一个新的 RSA 密钥对，并存储在部署 Sealed-Secret 命名空间下的 Secret 对象中，你可以通过下面的命令来查看。

 复制代码

```
1 $ kubectl get secret -n kube-system
2 NAME                                TYPE                                DATA  AGE
3 sealed-secrets-keyj78wj             kubernetes.io/tls                  2      131m
```

上面输出的结果中，以 sealed-secrets 开头的 Secret 对象就是存储 RSA 密钥的对象。

要获取 RSA 密钥，你可以进一步查看。

 复制代码

```
1 $ kubectl get secret sealed-secrets-keyj78wj -n kube-system -o yaml
```

当我们在本地使用 kubeseal 加密一个 Secret 对象时，kubeseal 会从集群内下载 RSA 公钥，并使用它来加密 Secret 对象，然后生成加密后的 SealedSecret CRD 资源，也就是 SealedSecret 对象。

当集群内控制器监听到有新的 SealedSecret 对象被部署时，它会使用集群内的 RSA 私钥来解密信息，并且在集群内重新生成 Secret 对象，以便提供给工作负载使用。

生产建议

由于 Sealed-Secret 的加解密过程需要使用 RSA 密钥对，而 RSA 密钥对又是在部署控制器时自动生成的，所以，你需要额外留意存储 RSA 密钥的 Kubernetes Secret 对象。

尤其是在需要进行集群迁移时，**你需要对它进行备份**，如果遗失，将无法对 Git 仓库存储的 SealedSecret 对象解密，你需要重新从原始的 Secret 对象那里生成加密后的 SealedSecret 对象。

要对已有的 RSA 密钥进行备份，首先你可以导出存储它的 Secret 对象，并保存为 backup-sealed-secret-rsa.yaml 文件。

 复制代码

```
1 $ kubectl get secret -n kube-system -l sealedsecrets.bitnami.com/sealed-secrets-k
```

接下来，将它重新部署到新的集群内。

 复制代码

```
1 $ kubectl apply -f backup-sealed-secret-rsa.yaml
```

然后，再部署 **Sealed-Secret 控制器**。此时，因为 kube-system 命名空间下已存在 RSA 密钥对，Sealed-Secret 会默认使用它作为解密的密钥，这样就完成了集群的迁移工作。

总结

这节课，我们学习了如何使用 Sealed-Secret 来加密 GitOps 工作流的 Secret 对象。

Sealed-Secret 主要包含两个部分，分别是 CLI 工具和运行在 Kubernetes 集群内的控制器。其中，CLI 工具负责加密原始的 Secret 对象并生成 SealedSecret CRD 对象，运行在集群中的控制器主要负责解密以及重新生成原始的 Secret 对象。

在示例应用中，我对两种类型的密钥进行了加密，它们分别是镜像拉取凭据和普通的 Kubernetes Secret 对象。在实际的项目中，这两种类型的密钥是经常会用到的。

在将密钥进行加密后得到 SealedSecret CRD 对象后，将它存放到 Git 仓库中便能够和 GitOps 工作流进行结合。需要特别注意的是，由于在 Sealed-Secret 控制器在首次部署到集群时会重新生成 RSA 密钥对，所以当你要对集群进行迁移时，务必要将存储 RSA 密钥的 Secret 对象进行备份，并将它部署到新的集群中。这样在安装 Sealed-Secret 控制器时便能够自动使用已有的 RSA 密钥，从而顺利执行解密过程。

此外，出于安全考虑，Sealed-Secret 控制器默认每 30 天会生成新的 RSA 密钥对，但旧的 RSA 密钥对并不会被删除，所以通过旧的 RSA 密钥加密的 Secret 对象依然可以被解密。

最后，如果你对密钥的安全性要求极高，并且为了满足安全标准而必须要使用第三方的密钥管理系统，例如 AWS Secrets Manager，那么你可以尝试使用 [🔗 External-Secrets](#) 的方案，这里就不再详细介绍了，感兴趣的同学可以尝试实践。

思考题

最后，给你留一道思考题吧。

在现存的应用场景下，我们需要加密的 Secret 对象在集群里往往已经存在了。

你可以动手模拟这个场景：先创建一个 Secret 对象部署到 Kubernetes 集群中，再将它加密并应用到集群内，随后查看 Sealed-Secret Pod 日志，这时候你是否能顺利生成 Secret 对象呢？

提示，你可以执行下面的命令来生成 Secret 对象并查看 Pod Sealed-Secret 控制器的日志。

 复制代码

```
1 $ echo -n admin | kubectl create secret generic mysecret --dry-run=client --from-
2 # 查看 Sealed-Secret 的日志
3 $ kubectl logs -l app.kubernetes.io/name=sealed-secrets -n kube-system
4 # 查看 Secret，检查 ownerReferences 字段即可判断 Secret 是否由 Sealed-Secret 生成
5 $ kubectl get secret mysecret -o yaml
```

最后，尝试为 Secret 对象增加 sealedsecrets.bitnami.com/managed: “true” 的注解，重新应用到集群，这次 Sealed-Secret 能否成功生成 Secret 对象呢？

欢迎你给我留言交流讨论，你也可以把这节课分享给更多的朋友一起阅读。我们下节课见。

AI智能总结

本文深入探讨了GitOps工作流中的密钥管理问题，并提出了解决方案。作者首先指出了Git仓库中存放的机密信息的安全性问题，包括镜像拉取凭据、外部数据库连接信息、第三方服务的API KEY等。针对这些问题，作者介绍了三种常用的密钥管理方案，重点介绍了Sealed-Secrets的工作原理和安装方法。在示例应用介绍中，作者展示了如何创建一个简单的Deployment工作负载，并介绍了如何使用kubeseal创建加密后的Secret对象。最后，作者指导读者如何将Sealed-Secrets和GitOps工作流结合，通过创建ArgoCD应用并将加密后的Secret对象推送到Git仓库中，实现了密钥的安全存储和部署。整体而言，本文深入浅出地介绍了GitOps工作流中的密钥管理问题及解决方案，对于需要提升GitOps安全性的读者具有一定的参考价值。文章内容详实，适合对GitOps工作流中的密钥管理问题感兴趣的读者深入学习。文章还提供了关于Sealed-Secret的生产建议和总结，以及一个思考题，为读者提供了更多的思考和实践机会。

© 版权归极客邦科技所有，未经许可不得传播售卖。页面已增加防盗追踪，如有侵权极客邦将依法追究其法律责任。

全部留言 (7)

最新 精选



Wang Yifei

2023-05-06 来自北京

这个方案仍然会生成在kubernetes中生成secret对象，有权限访问secret的人员仍然能得到密码，有更安全的方案么？

作者回复: 集群应该保护起来，不对外暴露。如果一定要实现不生成 Secret 的方案，那可能需要一套配置中心来处理。



1



po

2023-02-26 来自上海

kubeseal 需要连接到集群中，但是 kubeseal 平常都是开发或者运维在测试环境或者电脑本机，测试环境或者本机连接到生产kubernetes一般不大可能，现在想到的可能就是生产和测试kubernetes都同步一样的RSA，然后提交到Git，感觉这个管理流程上不是很顺畅，

作者回复: Kubeseal 的使用场景是可以直连 Kubernetes，本质上它需要把 RSA 秘钥保存到集群里，如果不能满足这个条件的话，那么只能按照迁移的模式提前生成好 RSA 秘钥，这可能不利于安全性的提升。

共 3 条评论 >

1



Sophia-百鑫

2023-08-18 来自上海

老师好，关于原理解析部分，描述和原理图有一点不一致的地方，具体是第2步：sealed secret controller 生成 RSA key 并保存在kube-system命名空间下的secret对象下。不应该是保存在secret-demo 命名空间下的secret对象下。请老师确认一下。

作者回复: RSA key 是存放在 kube-system，业务的 secret 对象和 CRD 是放在业务命名空间下。



Sophia-百鑫

2023-08-17 来自上海

老师好，2个问题需求求助您解答：

- 1.示例中，为了达到预期结果，除了要把 application.yaml中的 repoURL值改成自己的github 地址外。是否也需要把image-pull-secret.yaml 中的 .dockerconfigjson 值也改成自己的github username 和 PAT ？
2. 我完成2个加密文件后，secret-demo 空间的 pod 还是 imagepullbackoff。具体如下内容。是什么原因导致的或如何排查？

➔ kubernetes-example git:(main) ✕ kubectl get pods -n secret-demo

NAME	READY	STATUS	RESTARTS	AGE
sample-spring-65f7bbf4fd-l86gr	0/1	ImagePullBackOff	0	43h

作者回复: 是的，密钥也需要换成自己的。你可以用 kubectl describe pod xx 命令来查看拉取镜像的具体日志，一般是凭据错误或者镜像不存在。

共 2 条评论 >



无名无姓

2023-02-14 来自北京

安全性问题特别重要

作者回复: 是的，在 GitOps 中明文存储密钥是不可取的。



黑鹰

2023-02-13 来自北京

在云原生环境下，数据库访问凭证动态定期轮换也很重要!! Sealed-Secret 好像不支持该特性，Vault有比较好的支持！

作者回复: Vault 是一款老牌的密钥管理工具了，ArgoCD 可以结合 Vault 插件来使用。



Geek_45a572

2024-04-13 来自陕西

Secret 对象增加 sealedsecrets.bitnami.com/managed: “true” 的注解之后 需要重新加密后再应用到集群才会生效。 这里写的不太严谨

```
$ k edit secret mysecret
secret/mysecret edited
```

```
/opt/k8s/sealed-secrets 🖥 23:16:44
$ k apply -f admin-secret-sealed.yaml
sealedsecret.bitnami.com/mysecret unchanged
```

```
/opt/k8s/sealed-secrets 🖥 23:17:06
$ kubeseal -oyaml < ./admin-secret-existed.yaml > admin-secret-sealed.yaml
```

```
/opt/k8s/sealed-secrets 🖥 23:17:28
$ k apply -f admin-secret-sealed.yaml
sealedsecret.bitnami.com/mysecret configured
```

